

Rule Engine DSL: Documentation and Review

1. Introduction

This technical report details the implementation of a Rule Engine Domain-Specific Language (DSL). The project provides a lightweight, interpreted DSL written entirely in Python, using only the standard library. It is designed to evaluate conditional rules, manipulate local variables, and manage control flow structures such as branches and loops. Building a custom interpreter is highly appropriate for a Programming Languages laboratory, as it demonstrates fundamental compilation and evaluation phases without relying on code generation or Python's native `eval()` capabilities.

2. Laboratory Objectives

The project fulfills two primary laboratory requirements:

1. Construct a recursive Environment class with a parent pointer.
2. Build a tree-walk evaluator supporting conditional branches and loops.

The implementation addresses the first objective by using a recursive data structure to model lexical scoping. The second objective is met by implementing a visitor pattern that recursively traverses Abstract Syntax Tree (AST) nodes to execute statements and evaluate expressions in real time.

3. System Overview

The interpretation pipeline sequentially transforms raw source code into execution effects through several distinct phases. The architectural flow is as follows:

Source Code → Lexer → Tokens → Parser → AST → Tree-Walk Evaluator → Output

During execution, the Tree-Walk Evaluator communicates constantly with the recursive Environment. The Environment is responsible for storing variables and resolving names across nested scopes, ensuring that internal blocks can read outer variables and shadow them when necessary.

4. System Components

The interpreter is strictly divided into seven core Python modules:

- **main.py:** Handles file I/O, orchestrates the interpretation pipeline, configures the loop safety limit, and formats error reporting.
- **lexer.py:** Scans the source string to produce a stream of categorized tokens, recording line and column locations for diagnostics.

- **parser.py:** Consumes tokens to build an AST using a recursive-descent parsing strategy with operator precedence.
- **ast_nodes.py:** Defines immutable data classes representing specific expressions and statements within the tree.
- **environment.py:** Maintains state through a chain of dictionaries, managing both local variable bindings and parent scope delegation.
- **evaluator.py:** Recursively processes the AST to perform mathematical operations, evaluate boolean logic, and manage runtime control flow.
- **errors.py:** Defines domain-specific exception classes to halt execution upon encountering lexical, syntactic, or semantic faults.

5. Recursive Environment Implementation

The core of lexical scoping is managed by the Environment class. Every block of code (such as a loop body or conditional branch) instantiates a new Environment. This child environment maintains its own local variable dictionary and a parent pointer referencing the enclosing scope.

Variable resolution operates recursively: when the evaluator requests a variable, the Environment checks its local dictionary. If the name is absent, it recursively queries the parent environment. This permits inner scopes to access variables from enclosing scopes. Conversely, if an inner scope defines a variable with the same name as an outer variable, the inner definition shadows the outer one locally, leaving the parent unchanged.

```
class Environment:
    def __init__(self, parent: Environment | None = None)
    -> None:
        self.parent = parent
        self.values: dict[str, Value] = {}

    def get(self, name: str) -> Value:
        if name in self.values:
            return self.values[name]
        if self.parent is not None:
            return self.parent.get(name)
        raise EvaluationError(f"Undefined variable
        '{name}'".")
```

6. Tree-Walk Evaluator Implementation

The evaluator acts as an AST visitor. Expressions are evaluated recursively to produce typed values (booleans, numbers, or strings). Statements are executed for their side effects, such as printing output or mutating the environment.

Conditional branching is handled by evaluating the condition expression and routing execution to either the 'then' block or an optional 'else' block. Else-if chains are supported naturally by nesting

an 'if' statement inside the 'else' block. While-loops evaluate their condition prior to every iteration. To prevent accidental infinite loops, the evaluator enforces a safety limit of 10,000 body iterations per loop encounter, throwing a runtime error if exceeded.

Before executing a block, the evaluator creates a new Environment linked to the current one. Upon completion or failure, the evaluator strictly restores the original environment using a Python try-finally structure.

7. Rule Engine DSL Syntax and Execution

The DSL syntax is C-like, terminating statements with semicolons and grouping blocks with braces. It supports standard comparative operators and logical short-circuiting. Named conditional rules evaluate their condition upon encountering the rule block.

```
let score = 85;
rule passing_score when (score >= 75) {
    print("Student passed!");
}
```

Rules are checked immediately in the current scope. If the condition is true, the action block is executed exactly once. The DSL does not register rules for deferred execution, provide reactive recalculations upon variable mutations, or implement a multi-pass inference network.

8. Testing and Verification

The project utilizes Python's built-in unittest framework. Verification is executed using the command: `python -m unittest discover -s tests -v`. The latest test suite contains 124 successful tests, verifying parser logic, runtime safety limits, lexical scoping behavior, and diagnostic error generation.

Verification Category	Outcome
Recursive Environment Scoping	Passed
AST Operator Precedence	Passed
Control Flow and Loop Safety	Passed
Diagnostic Error Generation	Passed
Full Example Script Execution	Passed

9. Limitations

The interpreter's execution depth is strictly bound by Python's native call stack limit, meaning deeply nested source expressions may trigger a recursion error. The DSL currently supports dynamic, single-pass procedural execution and does not include user-defined functions, complex data structures like arrays, or object-oriented classes. Furthermore, the absence of forward and backward chaining precludes its use as a persistent expert system.

10. Conclusion

The custom Rule Engine DSL successfully fulfills the laboratory requirements. By constructing a recursive Environment and pairing it with a Tree-Walk Evaluator, the project concretely demonstrates the mechanics of lexical scoping, dynamic typing, and control flow execution. The resulting interpreter operates efficiently without external dependencies, accurately interpreting source programs and diagnosing syntactical errors, making it a complete educational model of a programming language runtime.